

GLASSES

Developer manual

SDK / Extensions / Plugins

Build useful local tools. Preserve captured evidence.
Design accessible interfaces. Ship compatible packages.

GLASSES 0.1.0 / EXTENSION API v1

Source-reviewed edition • 6 November 2026

Practical journeys + complete example + API reference + release checklist

About this edition

A practical implementation guide for Glasses 0.1.0 and Extension API v1.

This manual explains how to build, test and distribute local Glasses extensions. It covers declarative header rules, response observations, sidebar tools, context-menu actions and the browser-side host bridge. In this repository, **plugin** and **extension** describe the same package; **SDK** means the manifest contract, bridge and examples. There is no separately published SDK dependency to install.

Read by outcome

You want to...	Start with...
Build your first extension	Environment, first package and installation lifecycle.
Build a complete interactive tool	The JSON Workbench journey and its full source listings.
Integrate safely with traffic	Context, record, byte handling, bridge and request-draft reference.
Make a polished, portable tool	Coding standards, UI/UX, accessibility and compatibility chapters.
Review or release a package	Testing, troubleshooting, packaging and release checklist.

How to read requirements

Contract describes behavior enforced or implemented by this source snapshot. **Recommendation** is guidance proposed by this manual for robust extensions. **Boundary** identifies unsupported functionality or an implementation limitation. Recommendations do not imply a host validation rule.

> **Source baseline**
> Reviewed on 6 November 2026. package.json reports 0.1.0. The workspace has no Git commit yet, so the companion source fingerprint file identifies the reviewed files. The runtime validators take precedence over broad TypeScript types or older prose.

The examples use local lab data. They do not add a network API or scanner engine. Review destinations and credentials before sending any prepared request. The final chapters distinguish tests run for this manual from previously reported platform verification.

Contents / build and integrate

Click a topic or use your PDF viewer bookmarks.

Runtime and trust boundaries 5
Choose the smallest integration 6
Set up a local workspace 7
Your first complete package 8
Installation and tool lifecycle 9
Identity and strict validation 10
Collections and UI document 11
Capability reference 12
Invocation surfaces and meaning 13
Handle context as a snapshot 14
TrafficRecord reference 15
Bytes, text and compression 16
The complete bridge surface 17
RequestDraft validation 18
Header rule semantics 19
Response checks and findings 20
Local lab to an attributed finding 21
Build JSON Workbench 22
Workbench markup 23
Workbench styles 24
Workbench state and context 25
Workbench formatting and preview 26
Workbench request handoff 27
Build the installable artifact 28
Run the complete tool journey 29
From capture to a reviewed draft 30
Prepare an edited JSON request 31
Multiple tools in one package 32

Contents / design and ship

All page numbers refer to this PDF, including front matter.

Isolation is part of the contract 33
Secure coding for captured data 34
Language and code conventions 35
Modules, typing and packaging 36
Design rules for useful tools 37
Theme, tokens and brand 38
Accessibility acceptance rules 39
Empty, loading and failure states 40
Responsive layout and portability 41
Keep the interface responsive 42
Test behavior at the right layers 43
Schema and contract tests 44
Exercise the real host boundary 45
Compatibility matrix 46
Protocol and capture boundaries 47
Debug in a reproducible order 48
Symptom-to-cause reference 49
Data and rendering failures 50
Package and release responsibly 51
Versioning and maintenance 52
Where to extend the host 53
Limits and gotchas 54
Release acceptance checklist 55
Repository source atlas 56
External standards and guidance 57
Verification record and artifacts 58
Glossary and topic finder 59

Runtime and trust boundaries

Know where code runs before choosing an integration.

AUTHORING

Source assets + build-time Node

One validated JSON package

HOST

Main process + proxy core

Rules, checks, storage and request validation

APPLICATION

React workspace

Selection filtering + active-frame message checks

EXTENSION

Opaque-origin iframe

HTML, CSS, local JS + glassesPlugin bridge

The package installer validates and stores a manifest. Declarative rules and checks are interpreted by the host; their data is not arbitrary Node.js code. Only the ui.javascript string executes as extension code, inside the isolated frame.

The host can send a selected-context snapshot into that frame. The frame can request a text preview or validated Repeater draft. It cannot call the application preload API, enumerate the session, or send a request directly.

Implementation: core/plugins.cjs; core/plugin-document.cjs; src/PluginView.tsx; electron/main.cjs.

Choose the smallest integration

A capability is a specific permission, not a general-purpose API.

Need	Use	Avoid assuming
Set/remove a known header	Declarative rules + request.headers or response.headers	A callback on every request; dynamic body rewriting.
Report a response-header condition	checks + response.analyze	A custom severity field, URL-scoped checks or network probes.
Format or inspect selected text	UI + selection.read; optionally text.transform	Automatic access to the entire response.
Inspect one selected record	traffic.readSelected	A traffic subscription or query API.
Prepare an experiment	repeater.write	Automatic transmission or replay-job control.
Offer multiple screens	Several tools sharing one ui document	Separate per-tool packages or independent background workers.

Design a capability budget

Write down the input, output and user trigger for each feature. A selection-only JSON formatter needs ui.tools, ui.contextMenus, selection.read and text.transform. It does not need traffic.readSelected unless it displays or reuses request details. A rules-only package needs no HTML or JavaScript.

> **Current boundary**
> There is no plugin HTTP client, filesystem API, clipboard permission, settings API, persistent state API, asset directory, remote import, signed catalog, automatic updater or API-version negotiation in v1.

If a feature requires one of those facilities, reduce its scope or propose a host API change with explicit permission and lifecycle semantics. Do not reach into parent DOM, private IPC actions or the research browser to simulate the missing feature.

Set up a local workspace

Use the repository runtime to validate against the implementation you will ship with.

DEVELOPER COMMANDS

```
# From the Glasses repository root
node --version
npm ci
npm run dev

# Production UI and static checks
npm run build
npm start

# Repository verification
npm test
npm run test:ui
```

The repository selects Node.js 24 in .nvmrc and recommends it in the README. package.json accepts Node >=22.12.0; use the project-selected version for reproducible work. npm ci installs the lockfile versions. Development runs the native helper build before launching the app.

A separate extension source folder

COMPANION LAYOUT

```
docs/developer-manual/examples/
  manifest.json   # metadata and contributions, before UI assembly
  ui.html         # body markup
  ui.css          # local styles
  ui.js           # classic browser JavaScript
  build.cjs       # build-time Node script
  json-workbench.glasses-plugin.json  # installable output
```

You may use your own folder structure. The installation artifact remains one UTF-8 JSON file. Keeping source files separate improves review and avoids hand-escaping HTML, CSS and JavaScript inside JSON. The pre-build manifest is not installable until the builder attaches ui.

> **Build-time versus runtime**
> Node scripts can read local files during authoring. Installed extension JavaScript cannot use require, process, fs, Electron or the host window.glasses bridge. Bundle dependencies before packaging.

Your first complete package

Journey 1 starts with a declarative marker, with no UI code.

SAVE AS LAB-MARKER.GLASSES-PLUGIN.JSON

```
{
  "apiVersion": 1,
  "id": "org.example.lab-marker",
  "name": "Lab Marker",
  "version": "1.0.0",
  "author": "Example Developer",
  "description": "Mark local lab requests for review.",
  "capabilities": ["request.headers"],
  "rules": [{
    "id": "mark-local",
    "name": "Add local research marker",
    "enabled": true,
    "phase": "request",
    "match": "http://127.0.0.1:",
    "action": "set-header",
    "header": "X-Glasses-Research",
    "value": "manual-example"
  }]
}
```

This package requests only request.headers. Its rule uses a literal URL substring, so it matches HTTP requests to a loopback address with an explicit port. It does not match localhost, HTTPS, or every possible local address. Tailor the match to your controlled lab.

Use Plugins > Install from file, review the capability, install, then enable Lab Marker. Generate new traffic with Local lab. Inspect a fresh request: the header and applied-rule attribution should appear. Existing captures are not retroactively rewritten.

> **Acceptance condition**
> The rule has two switches: the installed plugin must be enabled, and the rule enabled field must be true. Disabling the plugin removes its effect on future traffic. A header modification is not a scanner finding.

Implementation: core/plugins.cjs: pluginSchema and applyRules; src/Extensions.tsx.

Installation and tool lifecycle

Installation is a deliberate, local workflow.

Stage	What the host does	What the developer should expect
Preview	Reads a JSON file, checks size and parses a strict schema.	Invalid fields or capability relationships fail before installation.
Install	Rejects an already-installed ID; stores the package disabled.	No rules, tools or menus become active until enabled.
Enable	Makes rules/checks available and adds declared UI contributions.	No extension background service starts.
Open	Serves the shared UI at its custom-scheme index document.	Register context handling immediately; display an empty state.
Reload tool	Remounts the frame using the stored installed manifest.	Memory resets. Editing your source file alone changes nothing.
Disable / remove	Removes active contributions and unloads the active tool.	Any unsaved frame state is lost.
Replace	Remove the old ID, then install the new file and enable it.	There is no in-place update or version-resolution mechanism.

The edit-test loop

Edit source, build the JSON, run schema and behavior checks, remove the installed package, install the new artifact, enable it, and exercise both sidebar and context-menu entry points. Record the package version and digest when testing; do not rely on a familiar sidebar label to identify a build.

Navigation away from the extension view can unmount its frame. Multiple sidebar tools inside one package share the same ui document and may reuse the same frame when switching tool IDs. Treat context changes as transitions, and reload as a complete reset.

Identity and strict validation

Contract: manifest objects reject unknown properties.

Field	Contract	Recommended practice
apiVersion	Exactly the number 1.	Keep this separate from your package version.
id	Max 100; lowercase alphanumeric segments joined by dot or hyphen; at least one separator.	Use a stable reverse-domain identity, such as org.example.json-tool.
name	Non-empty string, max 100.	Use a concise product name, not a capability list.
version	Digits.digits.digits only.	Use major.minor.patch; prerelease/build suffixes are rejected.
author	Required string, max 100.	Identify a publisher accurately; this is self-declared.
description	Required string, max 1,000.	Explain the outcome and sensitive data access plainly.
capabilities	Required array, at most 9 supported strings.	List each needed permission once; duplicates add no power.

EXAMPLE VALIDATE.CJS

```
// Validate with the exact host contract, from repository root.
const fs = require('node:fs');
const { pluginSchema } = require('./core/plugins.cjs');
const text = fs.readFileSync(process.argv[2], 'utf8');
if (Buffer.byteLength(text) > 256 * 1024) {
  throw new Error('Manifest exceeds 256 KiB');
}
const manifest = pluginSchema.parse(JSON.parse(text));
console.log(manifest.id, manifest.version);
```

The whole input file must fit 262,144 bytes at preview time. Whitespace and non-ASCII UTF-8 bytes count. Individual string limits use JavaScript string length, which is not the same as UTF-8 byte length. Always measure the final serialized file.

Implementation: core/plugins.cjs: pluginSchema, Plugins.preview and Plugins.install.

Collections and UI document

Contract: rules, checks, tools and contextMenus default to empty arrays.

Contribution	Required shape and limits
tools	At most 10. Each has id (max 50, lowercase a-z, digits or hyphen) and label (1-40 characters).
contextMenus	At most 30. Each has id (same rule as tool IDs), label (1-60), and contexts (1-9 supported surface strings).
ui	Required when tools or contextMenus are non-empty. html is required (max 100,000); css defaults to empty (max 50,000); javascript defaults to empty (max 100,000).
rules	At most 100 strict rule objects. A phase-specific header capability is required for every declared rule.
checks	At most 100 strict check objects. Non-empty checks require response.analyze.

Tool IDs and context action IDs must be unique across both lists in a package. Use letters, digits and hyphens for contribution IDs; dots allowed in package IDs are not allowed here. Do not reuse the same ID for a tool and its menu action.

One document, many entry points

The host inserts ui.html into the document body and ui.css into a style element. It inserts its bootstrap bridge before ui.javascript, which is a classic inline script. Supply body markup, not a second full HTML document. There is no relative asset routing beyond /index.html.

> **HTML parser boundary**
> JSON.stringify safely serializes JSON, but it does not neutralize HTML end tags after the host interpolates strings into script/style elements. Reject embedded script/style closing tags in those assets during the build, or use a reviewed context-aware bundling strategy.

Use ui.javascript for behavior, local CSS for appearance and data images for small static artwork. Remote fonts, stylesheets, scripts, images and module graphs are not a supported asset delivery mechanism.

Capability reference

Request the least data and authority your feature can use.

Capability	Enables	Boundary
ui.tools	Sidebar tool declarations.	Does not grant selected text or traffic.
ui.contextMenus	Context-menu contributions.	Needs ui and declared surfaces.
selection.read	Selected text in a context snapshot.	Capped at 1,048,576 string code units; may be empty.
traffic.readSelected	The selected TrafficRecord snapshot.	Includes sensitive headers/bodies; no session-wide list.
text.transform	showText result in the host preview.	Does not edit a capture; max 1,048,576 code units.
repeater.write	Validated request draft handoff.	User must click Send; no success callback.
request.headers	Declared request-header rules.	Transport-managed names are skipped at execution.
response.headers	Declared response-header rules.	No response body rewrite callback.
response.analyze	Declarative header observations.	Findings have review level; no arbitrary scanner code.

Capabilities are validated at installation and checked again at the host bridge. Undeclared selection/record data is omitted from context. Unauthorized text or Repeater messages are ignored. Declaring a permission does not guarantee there is a selected request or non-empty selection.

> **Review changes as behavior changes**
> Adding traffic.readSelected exposes headers, cookies and captured bodies to your tool. Treat a capability increase as a prominent release-note item. Do not request all nine permissions as a default template.

Implementation: core/plugins.cjs; src/PluginView.tsx.

Invocation surfaces and meaning

Contract: a contextual invocation opens the plugin pane.

Surface	Typical entry point / interpretation
traffic	Traffic row or a context that falls back to the general traffic surface.
request.headers / response.headers	Request or response header inspector. Selection may be only one value.
request.body / response.body	Text or byte inspector. Selected hex display text is not raw bytes.
repeater	Editable request context. Do not assume record contains unsent editor changes.
intercept	Intercepted request context. The plugin cannot forward/drop the pause.
timeline	Timeline or iteration-related context.
tls	Certificate-related context. This does not grant a full certificate-inspector API.

CONTEXT SHAPE - ILLUSTRATIVE JAVASCRIPT

```
{
  command: 'format-selection', // or 'open' from sidebar
  view: 'response.body',       // or 'tool' from sidebar
  toolId: undefined,           // sidebar contribution ID when present
  selection: '{"ok":true}',    // only with selection.read
  record: selectedRecord       // only with traffic.readSelected
}
```

The host uses the current selected record for context dispatch. The menu surface describes where the action was opened; it is not proof that record exactly represents the clicked row, an unsent Repeater editor or TLS probe state. Show the record method and URL before reuse, and test your intended surface.

A sidebar launch can include the selected record, but supplies no text selection. With selection.read the bridge normalizes the missing selection to an empty string. Unknown future view strings should fall back to a neutral view.

Implementation: src/App.tsx: menu dispatcher and enabledTools navigation; src/PluginView.tsx: push.

Handle context as a snapshot

Register once and make repeated delivery harmless.

PATTERN - IMPLEMENT RENDERSOURCE AND RENDERSELECTION IN YOUR TOOL

```
let current = {};
let previousKey = '';

function receive(context) {
  current = context;
  const key = JSON.stringify([
    context.command, context.view, context.toolId,
    context.record?.id, context.selection || '',
  ]);
  if (key === previousKey) return;
  previousKey = key;
  renderSource(context.record);  // textContent, not HTML
  renderSelection(context.selection || '');
}

glassesPlugin.onContext(receive);
if (window.glassesContext) receive(window.glassesContext);
```

The host sends context after its ready handshake and can push it again as the view updates. onContext appends a callback; it does not return an unsubscribe function and does not itself replay the last context. window.glassesContext holds the latest delivered object.

Be explicit about unsaved edits

Do not reset a user-edited input on an identical duplicate context. The small key above is suitable when only the selected text and record identity affect the editor. If your UI derives output from changing body/status fields, include those dependencies in your own update logic. There is no record revision number in v1.

A new context can replace the previous working set. A complex editor should show an in-frame choice to keep edits or load the new selection. A minimal tool can document replacement and keep a Clear action. Do not promise persistence across navigation or reload.

> **Avoid stale results**
> If an asynchronous operation is in flight, capture a local generation counter and compare it before rendering. Do not publish a result for an old selection into a newer context. Catch callback errors so one exception does not break your own UI lifecycle.

TrafficRecord reference

A structured capture observation, not an exact wire transcript.

Fields	Meaning and handling
id, number, startedAt	Stable record ID, session sequence number and ISO timestamp. Use id for identity; numbers are presentation.
url, method, protocol, source	Destination, method, protocol label and origin such as proxy or repeater. Treat labels as descriptive strings.
state, status, error, note	Lifecycle/status observations. status can be 0 before a response or on failure. Optional error/note need text rendering.
requestHeaders, responseHeaders	Record of header names to string or string[]. Preserve arrays, especially repeated Set-Cookie values.
requestBody, responseBody	Base64 strings of captured bytes. Empty strings are valid. They are not automatically decoded or decompressed for plugins.
requestBytes, responseBytes	Observed byte counts. A response total may exceed stored bytes when capture is truncated.
duration, timings	Milliseconds and optional proxy-observed phase timestamps: dns, connect, tls, firstByte, end.
appliedRules	Names of applied rules, with plugin names prefixed for attribution.
truncated, interceptPhase, tls	Optional response truncation flag, request/response pause phase, and TLS protocol/cipher/authorized observations.

Not every source populates every field. SOCKS captures can contain tunnel metadata without HTTP payloads; a selected request may still be pending; imported sessions have normalized data. Check presence and type before interpreting optional structures.

Treat received data as read-only. Mutating the context object does not update the host capture. Derive your own model and submit only supported bridge operations. Do not spread the entire record into a request draft: the draft is a separate validated contract.

Implementation: src/types.ts: TrafficRecord; core/engine.cjs: create, perform and replay; electron/main.cjs import handling.

Bytes, text and compression

Base64 is a transport representation, not a character encoding.

SMALL BOUNDED PAYLOAD HELPERS

```
function decodeUtf8Base64(base64) {
  // Apply a local length budget before atob allocates memory.
  if (base64.length > 512 * 1024) throw new Error('Too large');
  const bytes = Uint8Array.from(atob(base64), c => c.charCodeAt(0));
  return new TextDecoder('utf-8', { fatal: true }).decode(bytes);
}

function encodeUtf8Base64(text) {
  const bytes = new TextEncoder().encode(text);
  let binary = '';
  for (const byte of bytes) binary += String.fromCharCode(byte);
  return btoa(binary);
}
```

atob produces a binary string. It does not decode Unicode text. TextDecoder with fatal enabled reports invalid UTF-8 instead of silently substituting replacement characters. For other charsets, implement an explicit supported decoder choice and tell the user what was used. Preserve bytes for binary request replay. [W4]

Check Content-Encoding before text parsing

Captured gzip, deflate or Brotli bodies can remain compressed and retain content-encoding headers. Base64-decoding compressed bytes does not produce JSON. The host has its own bounded body decoder, but that internal IPC action is not exposed to extensions. Prefer a user-selected decoded text excerpt or report that the encoding is unsupported.

If you add local decompression, feature-test the browser API, bound both input and expanded output, handle malformed streams, and keep the UI responsive. Do not infer decompression support from one successful development-browser test.

> **Evidence integrity**
> A truncated response is a prefix, not a complete document. Refuse full-document claims and explain the limit. A base64 draft preserves captured request bytes; a utf8 draft intentionally encodes the supplied text. Neither mode reconstructs malformed HTTP wire framing.

The complete bridge surface

Contract: three methods and the last-context property.

API	Effect	Return / caveat
onContext(fn)	Registers a context callback.	No unsubscribe or automatic last-value replay.
showText(text)	Opens the host transform preview with a string.	Requires text.transform. No acknowledgement or Promise.
sendToRepeater(request)	Requests host validation and opens a request draft.	Requires repeater.write. It never sends automatically.
window.glassesContext	Latest received context object.	May be absent before first delivery; do not poll it as a traffic stream.

PUBLIC BRIDGE CALLS

```
// Only after an explicit user action and a successful transform.
glassesPlugin.showText(formattedText);

// A complete minimal draft. The user decides when to send.
glassesPlugin.sendToRepeater({
  url: 'http://127.0.0.1:9000/example',
  method: 'POST',
  headers: { 'content-type': 'application/json' },
  body: JSON.stringify({ example: true }),
  bodyEncoding: 'utf8',
});
```

The host accepts at most 20 recognized-marker messages per one-second reset window from the active frame. The ready handshake counts, and excess messages are silently ignored. This is a defensive ceiling, not a target send rate. Emit actions on intentional clicks; do not stream text previews on each keystroke.

showText accepts a string with at most 1,048,576 code units. A draft is validated in the main process before the app opens Repeater. Validation failures are shown by the host; v1 has no structured error result to await inside the extension.

Implementation: core/plugin-document.cjs: bridge bootstrap; src/PluginView.tsx: message listener.

RequestDraft validation

Prepare a clean request model and let the host manage transport headers.

Field	Enforced contract
url	Valid URL, max 16,384 characters, starting with lowercase http:// or https://. URL-embedded username/password are rejected.
method	1-20 uppercase A-Z letters. The validator does not restrict methods to GET/POST.
headers	Object of valid HTTP-token names to strings or arrays. Values max 16,384 each; arrays max 100 values; CR/LF/NUL rejected.
body	String, max 24 * 1,024 * 1,024 code units; defaults to empty.
bodyEncoding	utf8 or base64; defaults to utf8.

Draft validation is not a send guarantee. At replay time the engine applies the configured decoded-body limit, privacy bypass and routing checks. An oversized body may be accepted as a draft and fail when sent. A base64 string is not strictly validated as canonical base64 by the schema; validate your own bytes when correctness matters.

Headers and credentials

Remove Host, Content-Length and hop-by-hop transport headers when constructing a draft. The engine normalizes names, cleans connection headers, computes Host from the destination and recalculates request Content-Length. Preserve content-type and content-encoding only when they still describe the exact body bytes.

Authorization, Cookie and application API keys can remain in a copied request. Keep the original destination visible. If your feature changes the origin, require a deliberate review and avoid copying credentials to a different origin by default. Explain any headers you omit.

> **Honest action language**
> Use "Prepare in Repeater" or "Prepare original request". Do not label the button "Send", "Replay complete" or "Success". The bridge returns no success receipt and the network action remains under user control.

Implementation: core/engine.cjs: requestSchema, cleanHeaders, replay and perform; electron/main.cjs: validateDraft.

Header rule semantics

Contract: literal, case-sensitive URL-substring matching.

Rule field	Constraint / behavior
id / name	Required strings, each max 100. Use stable non-empty IDs/names as a convention; schema does not enforce uniqueness for rule IDs.
enabled / phase	Required boolean; phase is request or response. Matching phase capability is required even for disabled rules.
match	Required string, max 500. Empty matches every URL. No regex, glob, hostname parsing or templating.
action / header	set-header or remove-header. Header is a valid token name, max 128.
value	String max 8,192; defaults to empty; CR/LF/NUL rejected. Use empty for remove-header.

The engine executes host rewrite rules first, then enabled plugin rules in stored plugin order and manifest order. Later matching writes can replace earlier header values. There is no priority field or conflict-resolution API. Test interactions with other installed packages, and do not rely on another plugin being installed before yours.

> **Protected names are skipped at execution**
> host, connection, content-length, transfer-encoding, proxy-authorization and upgrade are skipped by applyRules. A manifest can pass schema validation while declaring one of these names; the runtime ignores that operation. Reject such rules in your own authoring checks to avoid misleading users.

Header names are lowercased when applied. A successful set/remove appends the rule name to appliedRules; plugin names are prefixed. Removing an already absent header can still be attributed as an applied rule. This provenance records matching operations, not a complete before/after diff.

Substring matching is not an authorization boundary. A host-like string can occur in a URL path or query. Use narrowly scoped lab values and review actual matched records. Plugin rules are also applied to Repeater traffic through the same engine.

Implementation: core/plugins.cjs: applyRules and Plugins.rules; core/engine.cjs: perform.

Response checks and findings

Contract: declarative observations over the stored response headers.

Check field	Constraint / behavior
header	Valid token, max 128; lookup uses its lowercase form.
when	missing, present or contains.
value	Max 500; defaults to empty; used only for contains.
title / advice	Required strings, max 150 and 1,000 respectively. Explain the context and an actionable next step.

missing means the looked-up value is undefined. present means it is not undefined, so an empty header still counts as present. contains uses String(value || "").includes(check.value), making the comparison case-sensitive. Arrays are string-coerced; this is not structured cookie or policy parsing.

> **Avoid an empty contains value**
> An empty search string matches every string, including the empty fallback used for an absent header. Always provide an intentional non-empty value for contains checks. The schema does not prevent this accidental broad match.

Each match produces a review-level finding with the check header, title, advice, evidence and plugin display name. Authors cannot set a custom severity, add a check ID or scope a check by URL in API v1. Checks have no per-check enabled flag; enabling the package activates its checks.

Observe carefully

Checks are evaluated when the host asks for header analysis. They inspect stored response headers, including relevant rewrite/interception changes, rather than a preserved pristine server header set. A missing request ID is an observability recommendation, not proof of a security vulnerability. State conditions under which an observation matters.

EXAMPLE ADVISORY CHECK; REQUIRES RESPONSE.ANALYZE

```
{
  "header": "x-request-id",
  "when": "missing",
  "title": "No request correlation ID",
  "advice": "Consider a request ID to aid service tracing."
}
```

Local lab to an attributed finding

Use the bundled Research headers example to complete the rule-and-check workflow.

1. Install a known example

Open Plugins and choose Header example to save the bundled manifest, or use plugin-sdk/examples/security-headers.json directly. Install it, review request.headers and response.analyze, and enable Research headers. Installation alone leaves it disabled.

2. Generate fresh local traffic

Choose Local lab. The host starts a loopback test server on an available port and generates real sample requests. Select /api/products. This is intentionally lab traffic; ordinary application startup does not seed fake requests. The example rule matches http://127.0.0.1: and adds X-Glasses-Research: local-lab.

3. Verify both outputs

Inspect the request headers and applied-rules list. Expect the marker and the attribution "Research headers: Mark local lab traffic". Open Header Hardening for that record. The local lab does not emit x-request-id, so the example check should produce the correlation-ID observation with the plugin name.

4. Exercise the negative paths

Disable the plugin and create a fresh lab request: the marker should no longer be added. Older records retain their captured headers. Re-enable it and try a URL that does not contain the literal match: the rule should not apply. The check has no URL scope, so it can still report a missing x-request-id on another selected response.

> **What this journey proves**
> You have tested installation, capability review, enablement, URL matching, header mutation, provenance, advisory findings and disablement. You have not tested arbitrary scanning, response-body rewriting or signed publishing, because those are not v1 facilities.

Troubleshoot by comparing the exact URL string, plugin/rule switches and new versus existing captures. If no finding appears, confirm the current selected record, the response header lookup and that response.analyze is declared.

Implementation: plugin-sdk/examples/security-headers.json; core/demo.cjs; tests/core.test.cjs.

Build JSON Workbench

A complete tool: selected text -> local formatting -> host preview.

JSON Workbench is the companion implementation for this manual. It opens from the sidebar with an empty editor, or from request/response body menus with selected text. It shows the selected request source, formats bounded JSON locally and exposes a separate action to prepare the original request in Repeater.

MANIFEST.JSON - UI ATTACHED BY BUILD.CJS

```
{
  "apiVersion": 1,
  "id": "org.example.json-workbench",
  "name": "JSON Workbench",
  "version": "1.0.0",
  "author": "Example Developer",
  "description": "Format selected JSON locally and prepare a reviewed request draft.",
  "capabilities": [
    "ui.tools", "ui.contextMenus", "selection.read",
    "traffic.readSelected", "text.transform", "repeater.write"
  ],
  "tools": [{ "id": "workbench", "label": "JSON Workbench" }],
  "contextMenus": [{
    "id": "format-selection",
    "label": "Format selected JSON",
    "contexts": ["request.body", "response.body", "repeater"]
  }]
}
```

The tool and menu IDs differ. The menu is deliberately limited to three useful surfaces. Removing the original-request handoff and source display would allow you to remove repeater.write and traffic.readSelected; formatting itself does not need traffic access.

> **Formatter scope**
> JSON.parse/JSON.stringify is a convenience formatter, not a lossless JSON editor. Large integers can lose precision; duplicate object keys collapse; number spellings and negative zero may change. It is unsuitable for signed JSON or byte-for-byte canonicalization without a dedicated parser.

Workbench markup

Complete ui.html. Keep captured content out of HTML construction.

UI.HTML

```
<main>
  <h1>JSON Workbench</h1>
  <p>Format a selection locally. Review the source before preparing a draft.</p>
  <p id="source">No selected request.</p>
  <label for="input">JSON input</label>
  <textarea id="input" spellcheck="false"
    aria-describedby="hint" placeholder='{"hello":"Glasses"}'></textarea>
  <p id="hint">Up to 256 KiB of UTF-8 input. Nothing is sent automatically.</p>
  <div class="actions">
    <button id="format" type="button">Format JSON</button>
    <button id="preview" type="button" disabled>Open transform preview</button>
    <button id="replay" type="button" disabled>Prepare original request in Repeater</button>
    <button id="clear" type="button">Clear input and result</button>
  </div>
  <p id="status" role="status" aria-live="polite">Ready for JSON.</p>
  <h2>Formatted result</h2>
  <pre id="output" tabindex="0" aria-label="Formatted JSON result"></pre>
</main>
```

Why these controls exist

The input has a visible associated label and a stable description. Primary actions use native buttons, so keyboard activation works without recreating browser interaction. Preview and Repeater actions start disabled until their preconditions are met. The result is a selectable, scrollable pre element with its own accessible name.

The short status region reports readiness, success or corrective guidance. Do not put a complete response body into an aria-live region; a screen reader would announce far too much. The result is separate from the small status message.

"Prepare original request" makes the data flow explicit: the formatted selection does not silently replace the captured request body. The Clear action clears only input and result; the selected request remains the handoff source, which the status message explains.

> **One authoritative source label**
> Show method and URL with textContent. This helps users catch a stale or unexpected selected record before preparing a draft. Do not infer the destination from highlighted JSON or a title string.

Workbench styles

Complete ui.css. These are extension-owned tokens, not host theme variables.

UI.CSS

```
:root {
  color-scheme: dark;
  --surface: #17191f;
  --raised: #22252e;
  --text: #e9ebef;
  --muted: #b4bac8;
  --accent: #ff7c83;
  --border: #697285;
}
* { box-sizing: border-box; }
body {
  margin: 0; padding: 24px;
  background: var(--surface); color: var(--text);
  font: 14px/1.55 system-ui, sans-serif;
}
main { max-width: 960px; margin-inline: auto; }
h1 { margin: 0 0 8px; font-size: 26px; }
h2 { font-size: 18px; }
p, #source { color: var(--muted); overflow-wrap: anywhere; }
label { display: block; font-weight: 600; margin: 20px 0 8px; }
button, textarea {
  font: inherit; border: 1px solid var(--border);
  border-radius: 7px; background: var(--raised); color: var(--text);
}
button { min-height: 40px; padding: 9px 12px; white-space: normal; }
button:disabled { opacity: .55; cursor: default; }
textarea {
  width: 100%; min-height: 180px; padding: 12px; resize: vertical;
  font-family: ui-monospace, monospace;
}
.actions { display: flex; flex-wrap: wrap; gap: 10px; margin: 16px 0; }
:focus-visible { outline: 2px solid var(--accent); outline-offset: 3px; }
pre {
  white-space: pre-wrap; overflow-wrap: anywhere; max-height: 420px;
  overflow: auto; padding: 16px; border: 1px solid var(--border);
  background: #101216; border-radius: 7px;
}
@media (max-width: 560px) {
  body { padding: 16px; }
  .actions { flex-direction: column; align-items: stretch; }
}
@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after { animation: none !important; transition: none !important; }
}
```

Workbench state and context

ui.js, part 1 of 3. Join all three parts in order into one file.

UI.JS / LINES 1-36

```
(() => {
  'use strict';
  const $ = (id) => document.getElementById(id);
  const input = $('input'), output = $('output'), status = $('status');
  const preview = $('preview'), replay = $('replay');
  const encoder = new TextEncoder();
  const INPUT_LIMIT = 256 * 1024;
  const RESULT_LIMIT = 512 * 1024;
  let context = {}, result = '', lastContextKey = '';

  function clearResult(message) {
    result = '';
    output.textContent = '';
    preview.disabled = true;
    status.textContent = message;
  }

  function receive(next) {
    const key = JSON.stringify([
      next.command, next.view, next.toolId,
      next.record?.id, next.selection || '',
    ]);
    context = next;
    replay.disabled = !next.record || !/^https?:\/\//.test(next.record.url || '');
    $('source').textContent = next.record
      ? `${next.record.method} ${next.record.url}`
      : 'No selected request.';
    if (key === lastContextKey) return;
    lastContextKey = key;
    input.value = next.selection || '';
    clearResult(input.value ? 'Selection loaded. Choose Format JSON.' : 'Ready for JSON.');
  }

  glassesPlugin.onContext(receive);
  if (window.glassesContext) receive(window.glassesContext);
  input.addEventListener('input', () => clearResult('Input changed. Format again.'));
```

An identical context leaves the user editor untouched. A new context loads its selection and clears any previous result. The current record is still refreshed before deduplication. No host state or selected capture is mutated.

The listener registers once, then consumes window.glassesContext if available. This supports both normal handshake delivery and tooling that starts after the most recent context arrived.

Workbench formatting and preview

ui.js, part 2 of 3. Input changes invalidate the prior output.

UI.JS / LINES 37-65

```
$('format').addEventListener('click', () => {
  clearResult('Formatting...');
  try {
    if (input.value.length > INPUT_LIMIT ||
        encoder.encode(input.value).length > INPUT_LIMIT) {
      throw new Error('Input exceeds the 256 KiB UTF-8 limit. Select less text.');
    }
    const value = JSON.parse(input.value);
    const formatted = JSON.stringify(value, null, 2);
    if (formatted.length > RESULT_LIMIT) {
      throw new Error('Formatted result exceeds the local size limit.');
    }
    result = formatted;
    output.textContent = result;
    preview.disabled = false;
    status.textContent = 'Formatted locally. Captured traffic is unchanged.';
  } catch (error) {
    // Do not echo parser messages: they can include sensitive input fragments.
    status.textContent = error.message.startsWith('Input exceeds') ||
      error.message.startsWith('Formatted result exceeds')
      ? error.message : 'Invalid JSON. Check quotes, commas and bracket pairs.';
  }
});

preview.addEventListener('click', () => {
  if (!preview.disabled && result) glassesPlugin.showText(result);
});
```

The local input budget is intentionally lower than the host selection ceiling. The first string-length guard avoids allocating a very large UTF-8 buffer; TextEncoder then enforces the exact byte budget. The output budget uses code units and stays below the host text-preview limit.

Parser errors do not echo captured fragments, and invalid input clears the earlier output. That prevents an old successful result from looking like the result of a new failed operation. Rendering uses textContent, so HTML-looking strings stay inert.

> **Performance limit**
> This small example still parses synchronously. The input budget bounds work but does not promise a time guarantee for every shape. More demanding tools need profiling, yielding between phases and explicit cancellation where feasible.

Workbench request handoff

ui.js, part 3 of 3. Preserve original bytes and keep transmission explicit.

UI.JS / LINES 66-94

```
  replay.addEventListener('click', () => {
    const r = context.record;
    if (!r || replay.disabled) return;
    const headers = Object.create(null);
    const omit = new Set([
      'host', 'content-length', 'connection', 'proxy-connection',
      'keep-alive', 'transfer-encoding', 'upgrade', 'te', 'trailer',
      'proxy-authorization', 'proxy-authenticate',
    ]);
    const connection = Object.entries(r.requestHeaders || {})
      .find(([key]) => key.toLowerCase() === 'connection')?.[1];
    String(connection || '').toLowerCase().split(',')
      .forEach((name) => omit.add(name.trim()));
    for (const [name, value] of Object.entries(r.requestHeaders || {})) {
      if (!omit.has(name.toLowerCase())) headers[name.toLowerCase()] = value;
    }
    glassesPlugin.sendToRepeater({
      url: r.url, method: r.method, headers,
      body: r.requestBody || '', bodyEncoding: 'base64',
    });
    status.textContent = 'Draft requested. Review destination and credentials in Repeater.';
  });

  $('clear').addEventListener('click', () => {
    input.value = '';
    clearResult('Input and result cleared. Source request is still selected.');
    input.focus();
  });
})();
```

The request body remains base64, so binary data and compressed request bytes are not silently re-encoded as text. Transport headers and connection-nominated hop-by-hop fields are omitted. Content-Encoding remains appropriate because the body is unchanged.

The header dictionary has a null prototype so unusual property names remain data. The tool preserves application credentials for the same selected URL; the Repeater review remains essential. The displayed status says "Draft requested" because no bridge acknowledgement is available.

Build the installable artifact

Complete build.cjs. Run it with Node in the source checkout.

BUILD.CJS

```
// Run from this directory: node build.cjs
// Build-time Node APIs are not available inside the installed extension.
const fs = require('node:fs');
const path = require('node:path');
const { pluginSchema } = require('../../../core/plugins.cjs');
const root = __dirname;
const read = (name) => fs.readFileSync(path.join(root, name), 'utf8');
const javascript = read('ui.js');
const css = read('ui.css');
// The host inserts these fields into raw script/style elements.
// Reject end tags; JSON escaping alone does not protect HTML parsing.
if (/<\/script\b/i.test(javascript) || /<\/style\b/i.test(css)) {
  throw new Error('Remove embedded script/style end tags from package assets.');
}
const manifest = pluginSchema.parse({
  ...JSON.parse(read('manifest.json')),
  ui: { html: read('ui.html'), css, javascript },
});
const json = JSON.stringify(manifest, null, 2) + '\n';
const bytes = Buffer.byteLength(json, 'utf8');
if (bytes > 256 * 1024) throw new Error(`Package is too large: ${bytes} bytes`);
fs.writeFileSync(path.join(root, 'json-workbench.glasses-plugin.json'), json);
console.log(`Validated package: ${bytes} bytes`);
```

FROM THE REPOSITORY ROOT

```
node docs/developer-manual/examples/build.cjs
# Install the generated file through Plugins > Install from file.
# Enable JSON Workbench after capability review.
```

The builder parses the final manifest using the host schema and checks the serialized byte size. It rejects embedded script/style end tags because the host places those assets inside raw HTML elements. Formatting and bundling happen before this step, never inside the installed sandbox.

The companion builder uses a repository-relative import of the runtime validator. If you move the source folder, update that path or provide an explicit path to your target Glasses checkout. No public npm SDK package is implied by this example.

Run the complete tool journey

Use this sequence as a manual acceptance test.

Step	Action	Expected result
1	Build, install and enable JSON Workbench.	A sidebar entry and three declared context surfaces become available.
2	Open from the sidebar before selecting a request.	Empty editor; source says no selected request; Repeater action disabled.
3	Paste {"city":"København","count":3}; choose Format JSON.	Readable Unicode and two-space indentation; preview enabled.
4	Choose Open transform preview.	Host preview displays the result; captured traffic remains unchanged.
5	Change input to invalid JSON and format.	Prior result is cleared; helpful status; preview disabled.
6	Generate Local lab traffic; select /api/products and a JSON body excerpt.	Right-click exposes JSON Workbench > Format selected JSON.
7	Invoke the menu action.	Tool opens with selected text and method/URL source.
8	Reload tool, navigate away, then reopen.	Memory resets as documented; there is no persistence promise.

Additional edge cases

Try HTML-looking JSON strings, empty input, deeply nested data, a selection above the local input budget, long unbroken strings and a narrow pane. Repeat the same context delivery while editing and verify the editor is not reset. Exercise all actions with Tab and Enter.

> **A successful format is not a fidelity guarantee**
> Preserve the original capture. Formatting is an interpretation that may alter numeric fidelity or duplicate keys. Do not automatically replace a request body with the result, especially for signed payloads.

From capture to a reviewed draft

Use explicit handoff to make a controlled experiment.

1 / CAPTURE

Local lab or authorized target

Select one request and the relevant text

2 / ANALYZE

Extension tool

Transform locally and explain the source

3 / PREPARE

Host Repeater draft

Validate destination, method, headers and bytes

4 / SEND

Explicit user action

User clicks Send; host records the new request

Generate Local lab traffic and select /api/products, then open JSON Workbench. Confirm the source URL. Choose Prepare original request in Repeater. Verify the method, headers and body before clicking Send. A new request record appears only after the user sends the draft. For a POST example, visit the lab root in the research browser: its page generates /api/cart with a JSON body.

The original capture stays unchanged. Compare the new response with the source record. Repeater uses the recording/rules engine and does not enter the interactive interception queue; it is not a bridge for plugins to control pending interception.

Implementation: src/App.tsx: sendDraft; core/engine.cjs: replay and perform; README.md.

Prepare an edited JSON request

Recipe: an explicit new text body, reviewed destination and intentional headers.

RECIPE - CALL ONLY FROM A CLEARLY LABELED USER ACTION

```
function prepareJsonPost(record, editedText) {
  if (!record) throw new Error('Select a source request first');
  const destination = new URL(record.url);
  if (!['http:', 'https:'].includes(destination.protocol)) {
    throw new Error('Only HTTP(S) destinations are supported');
  }
  // Validation only; avoid parse/stringify if byte fidelity matters.
  JSON.parse(editedText);
  if (new TextEncoder().encode(editedText).length > 256 * 1024) {
    throw new Error('Edited body exceeds this tool limit');
  }
  glassesPlugin.sendToRepeater({
    url: destination.href,
    method: 'POST',
    headers: { 'content-type': 'application/json; charset=utf-8' },
    body: editedText,
    bodyEncoding: 'utf8',
  });
}
```

This recipe deliberately starts a new header set and uses POST. It does not silently copy cookies, authorization, compression headers or the source method. Explain that choice in your UI and let the user add required credentials in Repeater. For a general-purpose editor, present method and destination controls rather than hardcoding POST.

Why body metadata must change together

If you decode a compressed body, edit its text and send plain UTF-8, remove Content-Encoding and stale checksums or signatures. Content-Length must reflect the new bytes and is managed by the host. Changing a body can invalidate authentication signatures even when the URL and visible JSON structure look unchanged.

> **Validate locally, then defer to the host**
> Local checks provide immediate feedback; host validation remains authoritative. A draft can still fail because of credentials in the URL, unsupported characters, privacy bypass, body limits or routing failures at send time.

Multiple tools in one package

Reuse one document without assuming one frame per tool.

CONTRIBUTION FRAGMENT - NOT A STANDALONE JSON DOCUMENT

```
// Manifest fragments; merge into an otherwise valid package.
"tools": [
  { "id": "inspect", "label": "Inspect data" },
  { "id": "compare", "label": "Compare text" }
],
"contextMenus": [{
  "id": "compare-selection",
  "label": "Use selection in compare",
  "contexts": ["request.body", "response.body"]
}]
```

ROUTING PATTERN - SUPPLY THE REFERENCED PANEL MARKUP

```
glassesPlugin.onContext((context) => {
  const compare = context.toolId === 'compare' ||
    context.command === 'compare-selection';
  document.getElementById('inspect-panel').hidden = compare;
  document.getElementById('compare-panel').hidden = !compare;
  // Update only the panel inputs this invocation owns.
  if (compare && context.command === 'compare-selection') {
    document.getElementById('compare-right').value =
      context.selection || '';
  }
});
```

Start with a single ui document containing both panels. Declare ui.tools and ui.contextMenus plus selection.read where needed. Sidebar launches use command "open" with a toolId; menu launches use the action ID and may have no toolId. Route using both fields.

Test Inspect -> Compare -> Inspect without leaving the extension view. Then navigate to Traffic and return. Decide which values persist only during that frame lifetime, and explain that reload or navigation can discard them. API v1 does not provide durable storage for a compare workspace.

> **Keep navigation simple**
> Use a small number of tools with clear outcomes. Internal tabs need keyboard and focus behavior; independent sidebar tools already have host navigation. Do not recreate a second full Glasses sidebar inside the frame.

Isolation is part of the contract

Design inside the sandbox, and keep data separate from authority.

Boundary	Implemented behavior
Frame	sandbox="allow-scripts" with an opaque origin; separate glasses-plugin:// document; no app preload bridge.
CSP	Denies default loads, connections, child frames, workers, objects, forms and base URL changes. Allows inline package scripts/styles and data images.
Application session	Blocks external HTTP(S), WebSocket and FTP requests except constrained host development traffic; denies permissions and external frame navigation.
Browser authority	No Node integration; no direct filesystem/shell/OS bridge; WebRTC constructors are disabled in extension documents.
Host messaging	Checks the actual active frame window; filters context and operations by capability; validates drafts in the main process.

The use of postMessage with "*" inside the supplied bridge reflects opaque-origin communication. The host checks the sender window. Use the supplied bridge instead of writing your own message transport or treating event.origin as a normal HTTPS origin.

A sandbox limits authority, but it does not guarantee that a plugin UI is truthful or that its code is cheap to execute. CPU and memory quotas are not implemented. Keep work bounded and make the source of each action clear. Publishers are self-declared and packages are not signature-verified.

> **Do not weaken the host to make a tool work**
> Never ask users to enable Node integration, disable web security or add allow-same-origin so a dependency can run. Replace the dependency or propose a narrowly scoped host feature. Electron recommends context isolation, sandboxing and narrow IPC interfaces. [W5, W6]

Implementation: core/plugin-document.cjs; src/PluginView.tsx; electron/main.cjs; SECURITY.md.

Secure coding for captured data

The extension is an inspector of untrusted input.

Prefer	Avoid	Why
textContent or textarea.value	innerHTML from traffic	HTML-looking evidence must remain inert.
JSON.parse with bounded inputs	eval, Function or script injection	Captured code is data, even inside a sandbox.
Null-prototype dictionaries / Map	Dynamic assignment into shared prototypes	Header and JSON keys can resemble built-in property names.
Explicit URL parsing	String concatenation for destinations	Avoid accidental credentials, scheme changes and malformed URLs.
Small operational status messages	Raw headers/bodies in diagnostic logs	Captures can include secrets and personal data.
Bundled reviewed dependencies	CDN imports or runtime downloads	The extension has no remote dependency channel.

Protect source integrity

Keep raw capture values separate from decoded or transformed results. Label whether text is a selection, decoded body or generated output. Never imply that a preview modifies evidence. When redacting a result, keep that as a separate transformation with a visible description; do not quietly redact the underlying capture.

Bound untrusted structures

Treat objects with own keys named __proto__, constructor or toString as data. Avoid recursive rendering without depth and node-count limits. Avoid unbounded regular expressions over attacker-controlled strings. Long arrays, nested objects and decompression expansion can consume resources even without network access.

> **Protect reports too**
> Bug reports should use minimal synthetic fixtures. Include package/host versions, the failing action and sanitized error details. Do not attach production cookies, Authorization headers, private CA keys or whole unreviewed session exports.

Language and code conventions

Recommendations aligned with the repository, with the runtime contract kept separate.

Area	Team convention
Formatting	Use Prettier. The repository specifies single quotes, trailing commas and printWidth 100. Use consistent two-space indentation in source examples.
JavaScript	Prefer const, explicit branches, small named functions and early validation. Avoid hidden mutation, global event-handler overwrites and clever coercion.
TypeScript	Use strict checking for authoring. The host targets ES2022 with DOM types; compile extension TS before packaging. Type annotations cannot ship in ui.javascript.
Naming	camelCase variables/functions, PascalCase types/components, UPPER_CASE constants, kebab-case contribution IDs and a stable package ID.
Errors	Handle expected invalid inputs locally. Distinguish missing context, unsupported encoding, parse failure and size limits; do not collapse every failure into "Something went wrong".
Comments	Explain data assumptions and boundary decisions. Avoid comments that simply restate obvious lines.
Dependency hygiene	Pin or lock build dependencies, inspect transitive code, record licenses and keep runtime bundles small.

Separate responsibilities

Keep pure transform functions independent of DOM and host messages. Let a small adapter read context, validate user input, update controls and call the bridge. This makes difficult parsing and byte-preservation logic testable without pretending that a mock browser reproduces the Electron sandbox.

Prefer deterministic transforms: identical inputs should produce identical outputs unless the feature explicitly involves time or randomness. Preserve original values and return new result objects. Do not change the global JSON, Array, Object or DOM prototypes.

Implementation: .prettierrc.json; tsconfig.json. The remaining conventions are author recommendations.

Modules, typing and packaging

Use a build system without inventing new runtime capabilities.

TYPE EXCERPT - NOT A COMPLETE STANDALONE DECLARATION FILE

```
// Authoring-only declarations; maintain against the host source.
type Headers = Record<string, string | string[]>;
type Draft = {
  url: string;
  method: string;
  headers: Headers;
  body: string;
  bodyEncoding: 'utf8' | 'base64';
};
type Context = {
  command: string;
  view: string;
  toolId?: string;
  selection?: string;
  record?: TrafficRecord; // copy the reviewed host definition
};
interface PluginBridge {
  onContext(callback: (context: Context) => void): void;
  showText(text: string): void;
  sendToRepeater(request: Draft): void;
}
```

Use types to catch authoring mistakes, but validate externally sourced values and keep host validation authoritative. src/types.ts uses broad fields such as apiVersion: number and capabilities: string[]; those types are more permissive than pluginSchema and are not a substitute for it.

Bundle to one browser-compatible classic script, usually an immediately invoked function expression. Avoid dynamic imports, runtime module fetching, eval-based development bundles, source-map fetches, worker entry points or Node polyfills that expect unavailable APIs. The host script element is not type="module".

> **Framework choice**
> Plain DOM code is often enough. React or another UI framework can be bundled locally, but its code counts toward the single 256 KiB package budget. The host using React does not expose React to your iframe. Measure the actual escaped JSON output, not just the uncompressed JavaScript file.

Design rules for useful tools

Recommendations for a coherent extension experience.

Lead with a task and its source

Use one clear title and a short description of what the tool does. Show the current request or selection source near the input. Place the primary operation beside its input, then display output and an intentional handoff action. Keep captured content visually distinct from instructions, warnings and generated results.

Make effects explicit

Use verbs such as Format JSON, Decode selection, Compare text and Prepare in Repeater. Say whether an operation changes a local editor, opens a preview or prepares a request. Do not use an ambiguous Run button for unrelated actions. Do not imply transmission when no request is sent.

Keep the host recognizable

Glasses already provides navigation, a tool header, capability badges and an isolation notice. Do not duplicate the entire application shell or imitate trusted host dialogs. Your own title can identify the working area, but avoid competing banners and repeated permission badges inside a small pane.

Explain decisions where they happen

Show a size limit near the input; explain an unsupported encoding near the decode control; show a source URL next to the request handoff. Inline guidance is easier to use than a long warning displayed after the user has already lost work.

Keep evidence readable

Use monospace for bytes, JSON and headers; normal text for explanations. Preserve selectable output and useful whitespace. Offer wrap or bounded scrolling for long lines. A red border alone does not explain a parse failure; add a short message and a recovery action.

> **A useful decision test**
> For each button, a new user should be able to answer: What data will it use? What will change? Where will the result appear? Will anything leave this pane? If any answer is unclear, improve the label or adjacent explanation.

Theme, tokens and brand

Contract: host CSS variables do not cross the isolated frame boundary.

Token / detail	Host reference	Extension recommendation
Surface / text	Host dark surfaces use charcoal; the plugin bootstrap defaults to #17191f and #e9ebef.	Define your own semantic surface, text, muted, border and accent tokens.
Accent	Host supports red, violet, ocean and amber palettes.	Use restrained accent color; do not depend on receiving the host palette.
Controls	Host uses small radii, clear borders, compact spacing and visible focus.	Use a 4/8 px spacing rhythm, roughly 6-8 px radii, and comfortably sized controls.
Type	Host uses system/Inter-style sans and monospace data views.	Use locally available system fonts; no remote font downloads.
Brand	THUGS uses theme foreground; red suffix and sunglasses stay red.	Use official assets accurately; do not make publisher identity look host-verified.

The host does not send theme, palette, density or platform through ExtensionContext. A CSS media query can detect OS color preference, but it cannot reliably reflect a host theme override. Do not promise automatic theme synchronization in API v1.

Choose an honest theme policy

A fixed dark tool is consistent with the bootstrap and simplest to maintain. A self-contained light/dark switch can be useful, but its choice is in-memory unless you have another supported persistence design. An OS-following theme should be labeled as OS-following. Test every palette you actually provide.

> **Contrast beats exact imitation**
> Host token values are a visual reference, not a certified accessible palette for every extension. Measure foreground/background combinations in your actual controls, status text and focus states. JSON Workbench uses a stronger muted color and border than the bootstrap for clarity.

Implementation: src/styles.css; core/plugin-document.cjs; docs/brand/README.md.

Accessibility acceptance rules

Use these as release criteria; the manifest validator does not test them.

Area	Recommended acceptance check
Keyboard	Reach every action with Tab; activate native buttons with Enter/Space; no keyboard trap in editors, tabs or output panes.
Focus	Every keyboard-focusable control has a visible, persistent indicator. Do not remove outlines without a replacement. Keep focus visible after errors and panel changes. [W3]
Names and structure	Associate labels with inputs; use headings in order; give icon buttons accessible names; use semantic buttons and table headers.
Text contrast	Target at least 4.5:1 for ordinary text and 3:1 for qualifying large text, subject to WCAG exceptions. Test the actual colors in both states/themes. [W1]
Pointer targets	WCAG 2.2 AA uses a 24 by 24 CSS pixel minimum with exceptions, including spacing. Prefer 40-44 px control heights where layout permits. [W2]
Status and errors	Use a short status region; identify the field and recovery step; do not communicate solely by color or announce entire payloads.
Zoom and reflow	Check 200% zoom, narrow pane widths, long text and operating-system text scaling. Allow vertical growth and avoid overlapping controls.
Motion	Avoid ornamental animation; respect prefers-reduced-motion for nonessential transitions.

A passing automated accessibility scan does not establish complete conformance. Add keyboard and screen-reader review for the actual workflows. The provided CSS has focus styling and semantic controls, but the manual does not claim a formal WCAG audit of Glasses or the example.

> **Do not steal focus on repeated context**
> Repeated snapshot delivery should not bounce focus out of an editor. Move focus only when it helps an explicit user action, such as Clear returning to the input or a validation summary directing the user to a field.

Empty, loading and failure states

Define states before adding more controls.

State	Visible behavior	Recovery
No selection	Describe what the tool accepts and where to invoke it; allow paste if meaningful.	Select text in a supported inspector or paste a small fixture.
No record	Keep text-only features usable; disable record-dependent handoff.	Select a traffic row and reopen the tool.
Ready / dirty	Show current input; invalidate stale derived output after edits.	Run the transform again.
Processing	Short progress text; disable repeat activation where needed.	Offer cancellation only if work can actually be stopped.
Invalid data	Clear or label stale output; identify the category of error.	Correct syntax, choose another encoding or select less data.
Unsupported capture	Explain binary/compressed/truncated limitations without pretending a full result.	Use a supported decoded selection or inspect raw bytes in the host.
Result ready	Show output provenance and meaningful next actions.	Preview, compare or prepare a reviewed draft.
Context changed	Replace or explicitly queue the new selection according to the documented policy.	Let users preserve edits if the workflow is complex.

Useful microcopy

Prefer "Invalid JSON. Check quotes, commas and bracket pairs." to an unexplained parser stack. Prefer "Input exceeds this tool's 256 KiB limit. Select less text." to a generic failure. Prefer "Draft requested. Review the destination in Repeater." to "Request sent".

Keep errors near the relevant input. If the host rejects a draft, it displays its own notification. Your extension should not claim host acceptance before that happens. Avoid rapid repeated submissions just because no callback arrived.

Responsive layout and portability

The application window is desktop-sized; your working pane can still be narrow.

LAYOUT RECIPE - CHOOSE WRAPPING APPROPRIATE TO THE DATA

```
.actions { display: flex; flex-wrap: wrap; gap: 10px; }
.field { min-width: 0; }
textarea, select { width: 100%; max-width: 100%; }
.source { overflow-wrap: anywhere; }
pre { white-space: pre-wrap; overflow-wrap: anywhere; }
@media (max-width: 560px) {
  .actions { flex-direction: column; align-items: stretch; }
}
```

Use CSS pixels and relative layout instead of platform-specific measurements. Avoid fixed panel heights that hide results, horizontal action strips that cannot wrap, and minimum widths larger than the iframe. Keep the primary action available when text grows or labels are localized.

Do not hardcode macOS-only shortcuts or assume Ctrl and Meta behave identically. Browser events inside an iframe do not necessarily trigger host shortcuts. Prefer visible actions and platform-neutral hints when the host exposes no platform API.

Data layout choices

For long tokens and URLs, allow wrapping or a deliberate horizontal scroll region with a visible full value. For byte grids, horizontal scrolling may preserve column alignment better than wrapping. For large tables, paginate or virtualize instead of building thousands of DOM nodes.

> **Test dimensions, not just platforms**
> Check a wide window, the smallest practical pane, 200% zoom and long unbroken values. The companion test probes a 420 px frame. This is a deliberately stressed layout width, not a guarantee about the host window's minimum size.

Use system fonts and test Unicode, emoji and combining characters as data. Count bytes when enforcing byte budgets, and use user-visible character language carefully: JavaScript string length counts UTF-16 code units.

Keep the interface responsive

The sandbox restricts authority; it does not provide a resource budget for you.

Risk	Recommended response
Large input	Set a tool-specific maximum below the host ceiling, validate before parsing, and explain the cap near the input.
Huge output	Cap expansion and render a preview or paginated result; do not append every object node into the DOM.
Repeated work	Debounce local editing when appropriate, deduplicate context handling and avoid re-parsing unchanged data.
Long computation	Split work into bounded chunks; yield with timers; track a generation token so stale work cannot overwrite a new context.
Compression	Cap expanded bytes, not just compressed input. Reject unsupported formats and truncated streams clearly.
Messaging	Send bridge actions only when intentional. Avoid approaching the 20-message-per-second ceiling.
Dependencies	Prefer a small parser/formatter over a complete IDE framework if the task does not need one.

PATTERN - SUPPLY BOUNDED TRANSFORMROW AND RENDERROWS FUNCTIONS

```
let generation = 0;
async function transformRows(rows) {
  const mine = ++generation;
  const result = [];
  for (let i = 0; i < rows.length; i += 200) {
    if (mine !== generation) return;
    result.push(...rows.slice(i, i + 200).map(transformRow));
    await new Promise(resolve => setTimeout(resolve, 0));
  }
  if (mine === generation) renderRows(result);
}
function cancelTransform() { generation++; }
```

Chunking only helps if each chunk is bounded. It cannot interrupt one large synchronous JSON.parse call. Workers are denied by the current CSP, so do not design cancellation around a worker that cannot start. Measure on a representative low-end machine before increasing limits.

Test behavior at the right layers

A valid manifest is the beginning of verification.

Layer	What to verify	Evidence to retain
Build / schema	Capabilities, unknown properties, IDs, field limits, byte budget, script/style end-tag checks.	Validation output and final package digest.
Pure logic	Unicode, malformed input, arrays, bytes, size limits, stable output and fidelity assumptions.	Small deterministic fixtures and assertions.
DOM behavior	Empty/dirty/error states, inert rendering, disabled actions and duplicate-context handling.	Behavior checks, not snapshots alone.
Real Electron host	Installed manifest, sandbox denial, permitted context, previews, draft validation and lifecycle.	Observed outcomes on the intended host build.
Human interaction	Keyboard, focus, zoom, source clarity and honest action labels.	Manual acceptance notes and screenshots.
Platform / package	OS, architecture, packaged runtime and host versions you support.	Explicit tested matrix; mark untested cells.

Use a clean test profile

The repository smoke harness creates a temporary user-data directory through GLASSES_TEST_DATA. The companion verifier does the same and uses an available loopback proxy port with SOCKS disabled for its own lab. It does not need to alter OS proxy settings or trust stores.

A harness may use host-only test APIs to seed fixtures or inspect state. That is test infrastructure, not an extension API. Never copy window.glasses.invoke or an Electron automation import into ui.javascript.

> **Negative tests matter**
> Verify that unauthorized capabilities fail, duplicate tool/action IDs are rejected, unsafe header values fail, and a draft does not create a new network request until the user sends it. A happy-path screenshot cannot establish those boundaries.

Schema and contract tests

Small tests for behavior that documentation commonly gets wrong.

EXECUTABLE COMMONJS TEST FROM REPOSITORY ROOT

```
const assert = require('node:assert/strict');
const { pluginSchema, applyRules } = require('./core/plugins.cjs');
const example = require('./plugin-sdk/examples/data-lens.json');

assert.ok(pluginSchema.safeParse(example).success);
assert.equal(pluginSchema.safeParse({
  ...example, capabilities: [],
}).success, false);
assert.equal(pluginSchema.safeParse({
  ...example, version: '1.0.0-beta',
}).success, false);
assert.equal(pluginSchema.safeParse({
  ...example, background: { script: 'worker.js' },
}).success, false);

const headers = { host: 'original.example' };
const applied = applyRules([{
  id: 'blocked-host', name: 'Blocked host edit', enabled: true,
  phase: 'request', match: '', action: 'set-header',
  header: 'Host', value: 'different.example',
}], 'request', { url: 'https://example.com/' }, headers);
assert.equal(headers.host, 'original.example');
assert.deepEqual(applied, []);
```

The final assertion demonstrates a runtime skip, not a schema error. Keep that distinction in your documentation and UI. Similarly, version strings with prerelease suffixes fail even if your release process otherwise follows semantic-versioning conventions.

For your own tool, test an empty context, a selection without a record, invalid JSON after a valid result, embedded HTML strings, multibyte UTF-8 near the input limit, and a record with array-valued headers. Assert that original input/capture data is not mutated.

Implementation: tests/security.test.cjs; tests/core.test.cjs; companion examples/verify.mjs.

Exercise the real host boundary

A practical browser-automation outline, plus the runnable companion harness.

OUTLINE INSIDE AN ALREADY-LAUNCHED TEST APP

```
// Developer automation with Playwright's Electron support.
const frame = page.frameLocator('iframe');
await frame.getByRole('textbox', { name: 'JSON input' })
  .fill('{"ok":true,"city":"København"}');
await frame.getByRole('button', { name: 'Format JSON', exact: true })
  .click();
await frame.getByRole('button', { name: 'Open transform preview' })
  .click();
await page.getByRole('dialog', { name: 'Transform selection' })
  .waitFor();
```

VERIFICATION COMMANDS

```
# Rebuild the tool and the host UI first.
node docs/developer-manual/examples/build.cjs
npm run build
node docs/developer-manual/examples/verify.mjs

# Existing project checks
node --test tests/core.test.cjs tests/security.test.cjs
npm run test:ui
```

The companion harness checks manifest rejection cases, protected headers, formatter behavior, Unicode, inert HTML, stale-output clearing, local limits, duplicate context, host preview, isolation, a narrow layout and the original-request handoff. It compares record counts before and after preparing a draft to detect accidental transmission.

Isolation checks cover missing Node/app bridges, denied parent DOM, denied localStorage and denied direct fetch. They exercise the current build, not every possible browser escape. Maintain the checks across Electron updates and add targeted regressions when the host bridge changes.

> **Do not overclaim**
> The manual's verification record lists the checks actually run for this edition. A documented npm command is not evidence it ran successfully. Platform packaging and a full accessibility review require their own explicit evidence.

Compatibility matrix

Declare tested combinations and explicit limitations.

Dimension	API v1 baseline	Release action
Host/API	Glasses 0.1.0, apiVersion exactly 1.	Validate against the host version you support; retain its source digest or commit.
Platforms	Web UI and manifest format are portable in design.	Test each OS/architecture you claim; do not infer runtime support from build configuration.
macOS	Release notes report local ARM64 desktop/package execution; x64 packages were built.	An x64 build alone is not an Intel hardware test.
Windows/Linux	Packaging targets exist; release notes do not report executed platform jobs.	Mark unverified until you exercise real packaged builds.
Themes	No extension theme/density context API.	Test your own theme policy next to host light and dark modes.
Dependencies	One classic browser script; no remote asset channel.	Bundle and validate the final JSON size.
Data	Optional context; base64 bodies; string or array headers.	Test empty, incomplete, binary, compressed and truncated cases.
Lifecycle	No persistent extension storage API or background runtime.	Test reload, navigation, disable, removal and replacement.

The repository describes platform packaging and earlier verification separately. This manual adds a current local test of the companion tool and records it in the verification appendix. It does not expand the platform support claim to systems that were not exercised.

> **A useful support statement**
> "Tested with Glasses 0.1.0 / API v1 on macOS ARM64. Fixed dark extension theme. Selected UTF-8 JSON up to 256 KiB. No persistence; no automatic sending." Replace these details with evidence from your own release.

Implementation: package.json; docs/RELEASE_NOTES.md; companion verification record.

Protocol and capture boundaries

Do not let a polished extension overstate what the host observed.

Host behavior	Implication for extensions
HTTP/1.1 inspection	HTTP/2, HTTP/3/QUIC and gRPC semantics are not implemented as dedicated inspection features.
WebSocket upgrades tunneled	Do not promise access to individual frames or frame-edit callbacks.
SOCKS5 TCP tunnel metadata	Do not assume an HTTP body exists for a SOCKS record; SOCKS UDP is unsupported.
Request body cap	Oversized requests are rejected according to configured limits; the plugin cannot raise the capture budget.
Response prefix capture	Large responses keep streaming while only a prefix is stored; whole-body pause/edit is unavailable for them.
Bounded session history	Records can be evicted by entry count and approximate stored-base64 memory budget. A context snapshot is not a session archive.
Imported sessions / HAR	Fields may be normalized, and original per-phase timings are not restored by HAR import.
Proxy timing observations	Do not label them browser rendering metrics, page load completion or exhaustive TLS performance.

The scope and privacy controls also matter. Privacy bypass is applied before record creation and HTTPS termination for excluded destinations, so those captures are not available to plugins. Existing captures and saved exports are not erased when privacy preferences change.

Header checks remain advisory. The TLS inspector observes an upstream connection and its certificate/trust details; it is not a complete cipher vulnerability scanner. Extensions should describe their evidence and limitations in the result, especially when data is incomplete.

Implementation: README.md; SECURITY.md; core/engine.cjs; core/formats.cjs; core/routing.cjs.

Debug in a reproducible order

Isolate manifest, lifecycle, input and bridge problems separately.

1. Identify the exact artifact

Record package ID/version, final file byte count and digest, host version, OS/architecture and the invocation surface. Confirm the installed package is the one you rebuilt. Remove/install is required for replacement; Reload tool only reloads the stored manifest.

2. Reduce to one small fixture

Use a tiny JSON string or a local lab request. If the issue occurs only for a production capture, reduce it into a synthetic fixture without secrets. Note whether the source is selected text, a raw base64 body or an edited request draft.

3. Confirm prerequisites

Check plugin enablement, declared capability, contribution ID, surface match and ui presence. For header rules, also check rule.enabled, phase and exact URL substring. For bridge actions, check the argument type and local size budget.

4. Observe the frame, then the host

Use developer tools or the development Electron harness to inspect console exceptions and CSP denials in the extension frame. Verify the frame has the expected context, but do not copy secrets into logs. Next inspect the host notification for rejected drafts and confirm no unsupported IPC access is involved.

5. Confirm recovery

A fixed failure should also recover without leaving stale output, disabled controls or a stuck loading state. Repeat it after reload and after a new selection. Record the smallest passing/failing steps so another developer can reproduce the problem.

> **Development versus packaged runtime**
> The packaged app uses hardened fuses and ignores the development-server environment flag. Use the normal Electron development runtime for deep automation, then separately verify the packaged application. A development bypass is not a supported extension feature.

Implementation: scripts/smoke.mjs; scripts/smoke-packaged.mjs; SECURITY.md.

Symptom-to-cause reference

Start with the likely cause, then verify the precise host behavior.

Symptom	Likely cause / check	Correction
Install rejected	Unknown property, missing capability/UI, duplicate tool/action ID, invalid version or oversized file.	Run pluginSchema on the final JSON; inspect structured validation errors.
Already installed	Same package ID already exists.	Remove it, install the new file, then enable.
No sidebar/menu	Plugin disabled; capability or contribution missing; current surface not declared.	Enable and test the exact intended surface.
No selected text	Sidebar launch, no actual selection, or selection.read missing.	Provide an empty state/paste flow; invoke from a supported text surface.
Wrong request source	Context uses the currently selected record; it may differ from unsent editor state.	Show method/URL; reselect the record or use a clearly defined manual input.
Network/storage fails	Sandbox/CSP denies the operation.	Use supported local computation and host handoffs.
New source not visible	Tool reload uses the stored manifest.	Rebuild, remove/install and enable the replacement.
Preview absent	Missing text.transform, non-string/oversized text or message-rate ceiling.	Validate and bound output; trigger once per intentional action.
Draft rejected	URL credentials, method/header validation or invalid draft shape.	Show a clean draft; examine the host error; do not retry in a loop.

Data and rendering failures

Failures often indicate a wrong data assumption rather than a missing permission.

Symptom	Interpretation	Better behavior
Base64 decodes to gibberish	Bytes may be compressed, binary or in another charset.	Check content-encoding/type and offer an explicit supported path.
JSON ends abruptly	Selection may be partial or the capture truncated.	Label partial input; ask for a complete supported selection.
Rule validates but has no effect	Protected transport header, mismatch or disabled rule/plugin.	Check runtime skip list and literal match; inspect fresh traffic.
Check appears everywhere	Contains value may be empty; checks have no URL scope.	Require a non-empty match and document package-wide analysis.
Result looks like active HTML	Untrusted data inserted using innerHTML.	Replace with textContent/value; add a regression with inert hostile strings.
Editor resets repeatedly	Duplicate context or callbacks registered multiple times.	Register once; deduplicate only the fields relevant to replacing input.
Result belongs to old selection	Asynchronous work races with a new context.	Use a generation token; discard stale completion.
Tool freezes	Unbounded parse/render/regex or large expanded data.	Reduce budgets, bound structure depth, paginate and profile.
Lab creates no captures	Listener conflict, routing/bypass failure or host-side error.	Check host notification and Settings; use a free test port. Never change OS proxy settings just to test a formatter.

Package and release responsibly

The release artifact is one local JSON manifest.

Prepare a reviewable release

Keep human-readable source files and a deterministic build script. Produce the final manifest, validate it with the target host schema and record a SHA-256 digest. Test the exact file that will be distributed. JSON whitespace counts toward size; minify only when needed, while retaining readable source for review.

Include useful accompanying information

- Purpose, publisher identity and contact channel; package ID/version and supported host/API versions.
- A plain-language explanation for every requested capability, including selected traffic and credential exposure.
- Installation, enablement, replacement and removal instructions; a small local acceptance journey.
- Known data limits, fidelity caveats, theme policy and supported platforms with testing evidence.
- Dependency/license notices, source location and release notes describing changes.

Do not promise a distribution feature that is absent

The community catalog/backend, signed publisher distribution and automatic updates are deferred. The local installer shows a self-declared publisher. A checksum helps identify a file; it does not by itself authenticate the publisher or establish that the file is safe.

> **Repository licensing**
> Glasses uses CC BY-NC 4.0 with an additional workplace and consulting permission. Non-commercial sharing and adaptation require attribution; commercial redistribution needs separate permission. Read LICENSE for the full scope and state your own extension package's terms clearly.

A release can remain entirely offline. Provide the manifest and supporting instructions through your established distribution channel; users install the file and review capabilities locally. Do not add a fake account or update flow to compensate for the missing catalog.

Versioning and maintenance

Stable identity, explicit upgrades and tested compatibility.

Change	Recommended handling
Bug fix	Increment patch; add the failing fixture or lifecycle case; retest the exact artifact.
Compatible feature	Increment minor; document new actions and any broader data access.
Breaking workflow/data change	Increment major; explain how outputs or user expectations change.
Capability increase	Highlight the new permission and why it is needed; re-run least-privilege review.
Host/Electron update	Revalidate schema and rerun actual sandbox, bridge, rendering and package tests.
Package rename	Keep the same ID when identity is unchanged; name is presentation, ID is installation identity.
Rollback	Remove the new version and reinstall a retained known-good JSON file; re-enable explicitly.

Version recommendations use major.minor.patch as a convention. The v1 validator enforces only the three numeric components; it does not compare versions, resolve dependencies, prevent downgrades or apply migrations. Do not include -beta or +build suffixes in version.

Treat the API as a versioned boundary

Keep assumptions centralized in your context adapter and manifest builder. When the host adds fields, tolerate them in received context, but do not add speculative fields to the strict manifest. There is no negotiated API revision or minHostVersion field in v1; maintain tested compatibility in accompanying release notes.

> **Future proposals are not present APIs**
> Persistent state, packaged assets, hook workers, integration brokers, signed catalogs and automatic updates are possible future work. Never ship code that depends on these as if they already exist. A proposal should define permission, trust, resource and lifecycle behavior before implementation.

Where to extend the host

This chapter is for Glasses maintainers, not installed-plugin runtime code.

Location	Responsibility / change impact
core/plugins.cjs	Manifest/rule schemas, storage, capability consistency, rule application and plugin findings.
core/plugin-document.cjs	Extension document template, CSP and browser bridge bootstrap.
src/PluginView.tsx	Frame lifecycle, context filtering, rate ceiling and allowed message operations.
src/App.tsx	Navigation, selected-record context, menu routing and Repeater handoff.
src/Extensions.tsx	Install review, enable/remove controls and example download actions.
electron/main.cjs	Custom protocol, session restrictions, trusted IPC dispatcher and main-process draft validation.
core/engine.cjs	Request validation, header cleanup, rule order, replay and capture semantics.
src/types.ts / tests / scripts	Authoring types, core/security tests and real desktop/package verification.

Design a new capability end to end

Specify the user outcome and minimum data exposure first. Define a bounded payload schema, host validation, explicit capability checks, failure behavior and lifecycle. Update the manifest validator, types, bridge, install-review copy, example and negative tests together. Check the active frame and trusted IPC sender at the relevant boundaries.

A new message type alone is not a complete API. Decide whether it returns an acknowledgement, what happens after unmount, how cancellation works, how resources are limited and how older plugins behave. Keep raw IPC and Node authority out of the extension frame.

Implementation: Named repository files; SECURITY.md.

Limits and gotchas

Keep this page next to your package builder.

Item	API v1 limit / fact
Final manifest file	256 KiB = 262,144 UTF-8 bytes, including whitespace.
Capabilities / collections	9 capability entries; 100 rules; 100 checks; 10 tools; 30 menu actions.
UI strings	html 100,000; css 50,000; javascript 100,000 code units; aggregate byte cap still applies.
Contribution IDs / labels	IDs max 50, a-z/0-9/hyphen. Tool label max 40; menu label max 60. IDs unique across both lists.
Selected text / text result	1,048,576 JavaScript code units each; selection is sliced, oversized text result ignored.
Bridge message rate	20 per one-second host counter window, including ready; excess ignored.
Draft URL / method	URL max 16,384; HTTP(S), no embedded credentials. Method 1-20 uppercase letters.
Draft headers / body	Each header value max 16,384; arrays max 100. Body max 24 Mi code units; decoded send limit is separate.
Rule header / value / match	Header max 128; value max 8,192; match max 500; literal URL substring.
Check fields	Value max 500; title max 150; advice max 1,000; contains is case-sensitive.
Persistent state / API negotiation	Neither is provided. One shared UI document; no background execution API.

String counts are not user-perceived characters and are not necessarily bytes. The SDK README uses "1 MiB" informally for selection/results, but PluginView enforces JavaScript string lengths. Build below all relevant ceilings and define smaller local budgets where work is expensive.

Implementation: core/plugins.cjs; src/PluginView.tsx; core/engine.cjs.

Release acceptance checklist

Recommendation: require evidence for each applicable item.

Area	Ready when...
Contract	Final JSON validates against the supported host; no speculative properties or permissions; package bytes measured.
Identity	Stable ID, valid numeric version, accurate publisher and a clear outcome-focused description.
Context	Sidebar and each menu surface work with empty/selected input; duplicate and changed contexts behave intentionally.
Data correctness	Unicode, binary, compression, truncation, arrays and precision limits are tested or clearly unsupported.
Bridge	Text stays bounded; invalid drafts are handled; preparing a draft never sends; action labels are truthful.
Security	Untrusted content is inert; no private IPC/network/storage assumptions; credentials handled intentionally; dependencies reviewed.
UI/UX	Source and effect are clear; errors recover; stale output is invalidated; layout survives narrow panes and long content.
Accessibility	Keyboard and focus paths checked; labels/names present; contrast measured; status announcements concise.
Performance	Input/output/structure limits defined; worst useful fixtures profiled; no repeated message flood.
Lifecycle	Reload, navigate, disable, remove, replace and rollback tested; persistence limitations explained.
Distribution	Exact tested file retained with digest, source, notices, capability rationale, install steps and compatibility matrix.

Review question

Can another developer reproduce the advertised result from a clean profile using only the delivered package and instructions? If not, complete the missing fixture, build step or explanation before calling the release ready.

Repository source atlas

Primary implementation evidence used throughout this manual.

Source	What it establishes
package.json; .nvmrc; .prettierrc.json; tsconfig.json	Host version, runtime/tool commands, build conventions and TypeScript settings.
plugin-sdk/README.md	Public API overview and intended extension isolation model.
plugin-sdk/examples/security-headers.json; data-lens.json	Bundled declarative and interactive examples.
core/plugins.cjs	Exact manifest limits and refinements; runtime rules/check behavior.
core/plugin-document.cjs	Document generation, CSP and complete public bridge methods.
src/PluginView.tsx; src/App.tsx; src/Extensions.tsx	Context delivery, host message limits, actual menu semantics and installation lifecycle.
src/types.ts; src/styles.css; docs/brand/README.md	Record/draft models, host design reference and brand conventions.
core/engine.cjs; core/formats.cjs; core/demo.cjs	Draft validation, byte capture, replay behavior, import/decode context and local lab.
electron/main.cjs; core/security.cjs; SECURITY.md	Privileged boundary checks, protocol serving and documented trust limits.
tests/core.test.cjs; tests/security.test.cjs; scripts/smoke.mjs	Core/security assertions and real Electron isolation/interaction examples.
README.md; docs/RELEASE_NOTES.md	Operating workflows, protocol boundaries and previously reported platform verification.

All paths are repository-relative. This edition treats implementation as authoritative where shorthand documentation differs, such as string-length limits, protected-header runtime skips and context snapshot semantics. See source-fingerprints.json beside the authoring source for full SHA-256 hashes.

External standards and guidance

Primary technical references checked on 6 November 2026.

W1 / W3C: Contrast (Minimum). Supports the ordinary-text 4.5:1 and qualifying large-text 3:1 targets, including exceptions. Read the W3C explanation.

W2 / W3C: Target Size (Minimum). Explains WCAG 2.2 AA's 24 by 24 CSS pixel minimum and its exceptions. The 40-44 px control-height preference in this manual is an ergonomic recommendation, not a restatement of that minimum. Read target-size guidance.

W3 / W3C: Focus Visible. Explains visible keyboard focus and its relationship to other focus/contrast requirements. Read focus guidance.

W4 / MDN: TextDecoder. Documents byte-to-text decoding and decoder behavior. Use the actual encoding and explicit error handling; base64 is not a text charset. Read TextDecoder documentation.

W5 / Electron: Security. Framework guidance on sandboxing, permissions, navigation, IPC validation and keeping privileged APIs away from untrusted content. Read the Electron security checklist.

W6 / Electron: Context Isolation. Explains the separation of preload/internal contexts from page code and the need for narrow exposed APIs. Read context-isolation documentation.

> **How these references are used**
> External guidance supports the recommended engineering and accessibility practices. It does not prove that Glasses or an extension satisfies every framework recommendation or accessibility criterion. Glasses-specific claims come from the reviewed repository and local verification.

Links are active in the PDF. For offline use, the requirements and practical implications needed for the tutorial are described in the relevant chapters; external pages provide further rationale and full exceptions.

Verification record and artifacts

Evidence for this manual edition, separate from the repository release history.

Check	Result for this edition
Core/security tests	23 passed using node --test tests/core.test.cjs tests/security.test.cjs.
Host build	npm run build passed TypeScript checking and the production Vite build.
Companion package	Built and parsed by the actual pluginSchema; serialized size 7,013 bytes before any future edits.
Companion Electron journey	Passed formatter, Unicode, inert HTML, invalid/oversized input, stale output, duplicate context, preview, sandbox, narrow layout and Repeater handoff checks.
Transmission boundary	Record-count assertion confirms that preparing the selected request draft did not create a new network request.
Local platform	macOS ARM64 development Electron, using a temporary profile. The execution shell reported Node 23.6.0; project-recommended Node 24 was not substituted for this run.
Not established here	Windows/Linux/Intel execution, notarized distribution, independent security review, exhaustive performance testing or formal accessibility conformance.

The first companion run reached the local-lab stage but did not create the expected row with default listener settings. The harness was isolated onto an available loopback proxy port with SOCKS disabled, then passed. The cause was not established as an application defect. The test changes affect only its temporary profile.

Companion files

docs/developer-manual contains the PDF builder, authored content, layout report and source fingerprint inventory. Its examples folder includes the complete JSON Workbench source, builder, installable JSON and developer-only Electron verifier. The archive delivered with the manual preserves this structure.

> **Rebuild and recheck**
> Run build.cjs after editing the example; run the host build and verify.mjs for behavior; rebuild the PDF with build_manual.py. Re-render and inspect the PDF after content changes. Source hashes identify the reviewed baseline; they do not authenticate a publisher.

Glossary and topic finder

Use PDF bookmarks or text search for exact API names.

Term	Meaning / where to look
Capability	A declared permission for one contribution, data class or bridge operation. See Capability reference.
Context	One invocation snapshot, including command/view and permitted selection/record. See Handle context as a snapshot.
Contribution	A tool or context-menu item declared by a package. IDs are unique across both lists.
Draft	A validated request model prepared for Repeater; it is not a transmitted request.
Finding	A header observation; plugin checks emit review-level advice. See Response checks and findings.
Host	The Glasses application, renderer and privileged runtime that manage data and extension permissions.
Manifest	The single UTF-8 JSON installation artifact, including optional embedded UI strings.
Opaque origin	An isolated frame origin that cannot access the parent application DOM as a same-origin page.
Provenance	Attribution explaining where data or a rule effect came from. appliedRules records names of matched operations.
Snapshot	Data captured at an invocation/update point; not a live traffic subscription.
Truncation	A capture contains only a prefix of response bytes. See Bytes, text and compression.
UTF-8 / base64	A text encoding versus a byte representation. Choose the correct RequestDraft bodyEncoding.

Find by task: Install or replace: Installation and tool lifecycle. Fix a rejected manifest: Identity and strict validation. Design a UI: Design rules for useful tools. Handle secrets: Secure coding for captured data. Ship a release: Package and release responsibly. Review exact limits: Limits and gotchas.